

NEXACORE

ISP BILLING & NETWORK OPERATIONS PLATFORM Technical Design Specification

Multi-tenant SaaS • PPPoE • Hotspot • Static IP • DHCP • M-PESA • RADIUS • MikroTik

Document field	Value
Version	2.0
Status	Updated target architecture and implementation blueprint
Date	18 September 2026
Prepared by	Pauline Moraa
Primary backend	Python + Flask
Database	PostgreSQL
AAA	FreeRADIUS
Network enforcement	MikroTik RouterOS and provisioning adapters
Payments	Safaricom Daraja / M-PESA STK Push
Client applications	React web admin; captive portal; future mobile app
Commercial model	Multi-tenant SaaS for ISPs and managed Wi-Fi operators

Purpose: A living engineering specification for building, selling, operating, securing, and extending Nexacore as a commercial platform used by multiple independent Internet service providers. The document supersedes the single-ISP assumptions of version 1.0 while preserving the proven Flask, PostgreSQL, FreeRADIUS, MikroTik, M-PESA, and shared billing-core direction.

Document Control

Revision	Date	Summary
1.0	16 Sep 2026	Baseline architecture for PPPoE + Hotspot billing, FreeRADIUS, MikroTik, M-PESA, APIs, security and deployment.
2.0	18 Sep 2026	Reframed Nexacore as a multi-tenant ISP SaaS platform; added Organization isolation, ISP vs platform roles, Static IP and DHCP access types, richer network/accounting analytics, dashboard requirements, phased OLT/CPE/FUP/Burst/support capabilities, and current implementation status.

Contents

- 1. Executive Summary and Product Vision
- 2. Product and Commercial Model
- 3. Current Implementation Status
- 4. Scope and Release Strategy
- 5. Architecture Principles
- 6. Target System Architecture
- 7. Technology Stack
- 8. Multi-Tenancy and Organization Model
- 9. Identity, Roles and Authorization
- 10. Domain Model and Data Architecture
- 11. Customer Management
- 12. Access Services: PPPoE, Hotspot, Static IP and DHCP
- 13. Sites, Routers and Network Inventory
- 14. Service Plans, Subscriptions, FUP and Burst
- 15. FreeRADIUS and AAA
- 16. MikroTik and Provisioning Integration
- 17. Sessions, Accounting and Traffic Analytics
- 18. M-PESA and Payments

- 19. Finance, Reports and Revenue Analytics
- 20. ISP Dashboard and User Experience
- 21. Platform Administration
- 22. Support, Communication and Operations
- 23. Flask Backend Architecture
- 24. API Design
- 25. Background Jobs and Automation
- 26. Security and Tenant Isolation
- 27. Reliability and Failure Handling
- 28. Observability, Auditing and Health
- 29. Backup and Disaster Recovery
- 30. Deployment and Environment Strategy
- 31. Testing Strategy
- 32. Performance and Scaling
- 33. Implementation Roadmap
- 34. Acceptance Criteria
- 35. Risks and Open Decisions
- Appendices A-H

1. Executive Summary and Product Vision

Nexacore is not only a Wi-Fi billing application. The target product is a commercial ISP billing, subscriber management, network access, payment, monitoring, and operations platform that can be sold to many Internet suppliers. Each ISP operates its own isolated organization inside the platform and manages its own customers, routers, service plans, network access accounts, subscriptions, payments, usage, and reports.

The platform supports multiple access methods under one commercial core: PPPoE, Hotspot, Static IP, and DHCP. PPPoE and Hotspot rely strongly on FreeRADIUS AAA; Static IP and DHCP can use router provisioning adapters and policy enforcement appropriate to each network design. All access types share customers, service plans, subscriptions, payments, reporting, tenant isolation, and audit services.

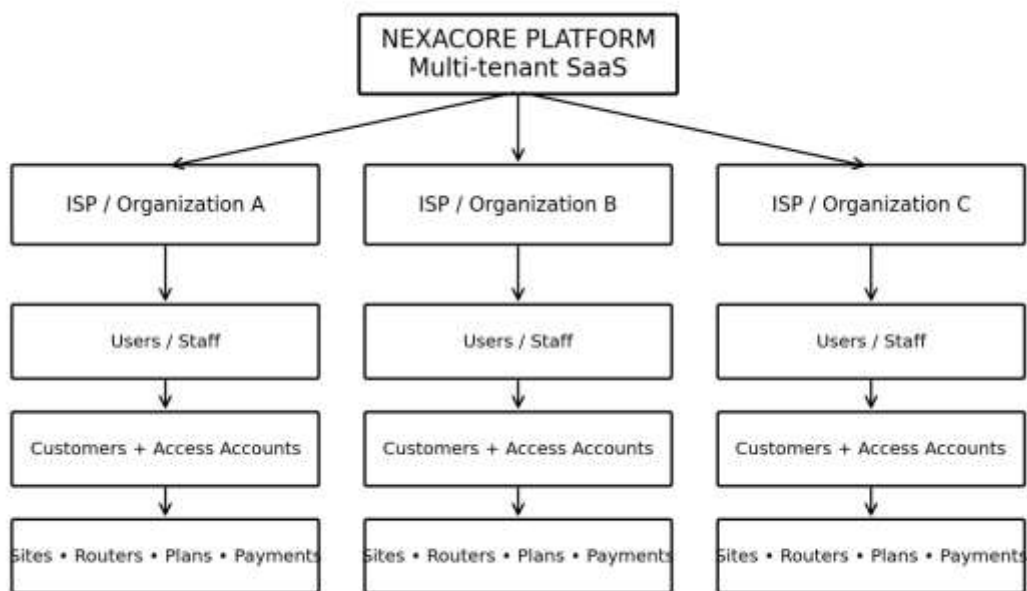


Figure 1. Nexacore commercial model: one SaaS platform serving multiple isolated ISP organizations.

The strategic separation remains: Flask owns business rules; PostgreSQL is the authoritative system of record; FreeRADIUS handles AAA and accounting where applicable; MikroTik and other network systems enforce access; M-PESA provides verified payment events; background workers execute retries, expiry, reconciliation, notifications, and usage aggregation.

2. Product and Commercial Model

2.1 Who buys Nexacore?

The direct Nexacore customer is an ISP, WISP, hotspot operator, apartment Wi-Fi business, estate network, school, hotel, cyber business, or managed connectivity provider. That organization then uses Nexacore to serve its own Internet subscribers.

2.2 SaaS tenancy

The preferred commercial architecture is multi-tenant SaaS. A single Nexacore deployment can host multiple organizations while enforcing strict tenant isolation. Optional branded subdomains may be introduced later, but the first implementation can use one application URL and derive organization context from the authenticated user.

Layer	Examples	Ownership
Nexacore platform	Organizations, platform subscriptions, platform health	Nexacore operator
ISP organization	Staff, sites, routers, plans, payment configuration	One ISP only
ISP subscriber domain	Customers, access accounts, subscriptions, sessions, payments	One ISP only
Network enforcement	MikroTik, RADIUS, OLT/CPE integrations	Scoped to ISP/site/router

2.3 Platform subscription vs Internet service plan

Two concepts must remain separate. A Platform Subscription is what an ISP pays Nexacore for use of the software. A Service Plan is what that ISP sells to its own subscribers, such as 5 Mbps monthly PPPoE or a 24-hour Hotspot package. The database, APIs, UI labels, and reports must not mix these concepts.

3. Current Implementation Status

As of 18 September 2026, the backend foundation is operational in local WSL development. This section records the as-built state so the design remains aligned with the codebase.

Area	Status	Notes
Flask application factory	Complete	create_app(), configuration, extensions and blueprints are working.
PostgreSQL	Complete	billing_system database connects through psycopg / SQLAlchemy.
Alembic / Flask-Migrate	Complete	admin_users and customers migrations applied.
JWT authentication	Complete	register, login, current-user and refresh flows established.
Authorization	Complete for current phase	super_admin-protected endpoints; two developers are platform super admins.
Customer model/API	Complete baseline	Create, list, search, filter, get, update and status update tested in Postman.
Multi-tenancy	Next critical milestone	Organization model and tenant isolation must be added before Sites/Routers.
Sites / Routers / Plans	Planned	Begin only after tenant model is in place.
RADIUS / MikroTik / M-PESA	Planned	Architecture defined; implementation follows core domain models.

3.1 Current backend layout

```

backend/
app/
    __init__.py
    authz.py
    config.py
    extensions.py
    api/
        __init__.py
        auth.py
        health.py
        customers.py
        models/

```

```

__init__.py
admin_user.py
customer.py      services/
__init__.py
migrations/
tests/      wsgi.py

```

3.2 Immediate architecture correction

The current Customer table does not yet identify which ISP owns each customer. Before more business tables are added, Nexacore must introduce Organization and organization_id scoping. Retrofitting tenancy after dozens of tables would be substantially riskier.

4. Scope and Release Strategy

4.1 Commercial MVP

- Organization/ISP onboarding and tenant isolation.
- Platform super-admin and ISP owner authentication.
- Customer management.
- Sites and MikroTik/NAS router inventory.
- Service plans and subscriptions.
- PPPoE access accounts and RADIUS authentication/accounting.
- Hotspot access accounts and captive-portal payments.
- Static IP assignments and DHCP/reservation-based access where appropriate.
- M-PESA STK Push with verified callback/reconciliation.
- Automatic activation, expiry, suspension and restoration.
- Online session view and traffic accounting.
- Revenue, payments, subscriber and network-health dashboards.
- Audit logs and basic operational reports.

4.2 Later operational capabilities

- Tickets and escalation.
- SMS/email communication and templates.
- FUP policies and burst profiles.
- Inventory and expenditure tracking.
- Sales commissions.
- OLT management.
- TR-069 / CPE management.
- Downtime compensation.
- Advanced analytics and custom dashboards.

4.3 Deliberately deferred

- Full ERP/general ledger.
- Complex tax engine and multi-currency accounting.
- Carrier-grade BRAS replacement.
- Automated physical fiber provisioning.
- Native customer mobile application as a launch dependency.

5. Architecture Principles

Principle	Implementation rule
Tenant isolation	Every tenant-owned business row carries organization_id; every tenant API query is scoped server-side.
Single source of truth	PostgreSQL owns business state. Router-local users and leases are not the billing database.

Access abstraction	Customer billing is not designed around PPPoE only; AccessAccount represents PPPoE, Hotspot, Static or DHCP.
AAA separation	FreeRADIUS handles AAA/accounting where applicable; Flask controls commercial entitlementment.
Verified payment finality	An STK initiation response never activates service; only verified success or reconciliation can finalize payment.
State separation	Customer, access account, subscription and session statuses are distinct.
Idempotency	Callbacks, renewals, provisioning actions and disconnects are safe to repeat.
Auditability	Every material payment, subscription, tenant, router and operator action is traceable.
Progressive delivery	Ship a coherent ISP lifecycle before adding OLT/CPE/ERP-grade extras.

6. Target System Architecture

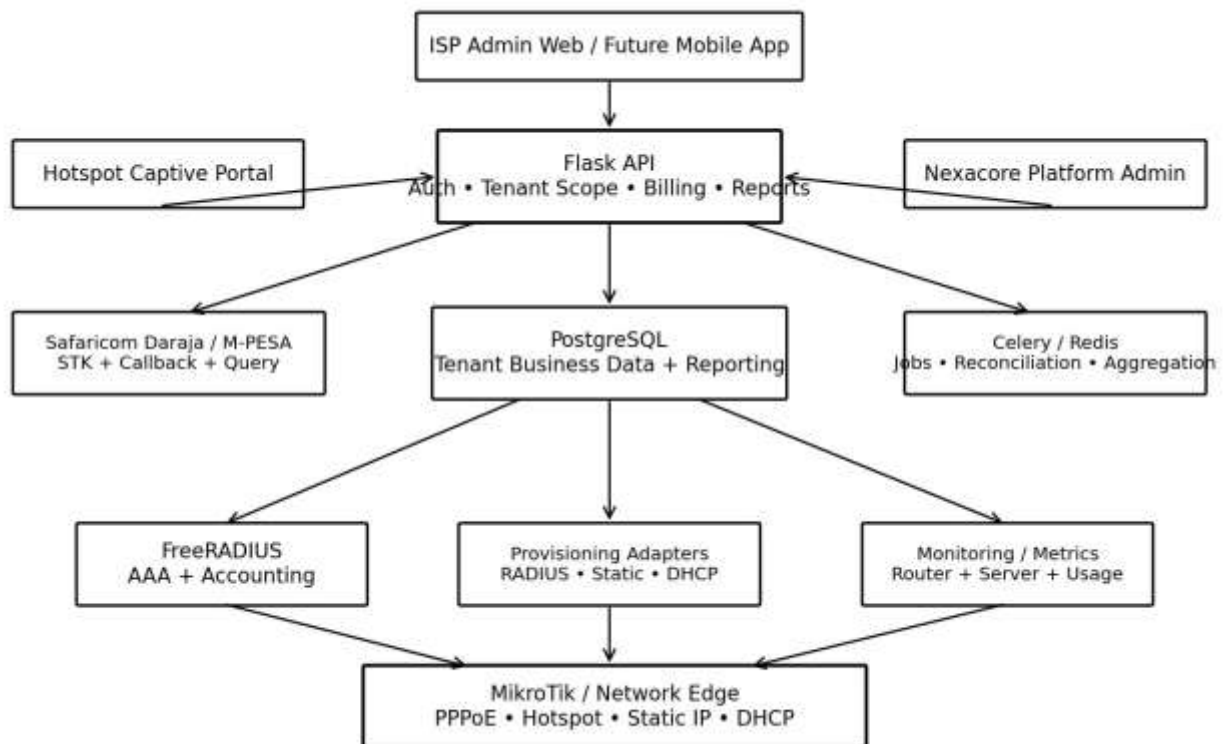


Figure 2. Target multi-tenant Nexacore architecture.

6.1 Component responsibilities

Component	Responsibilities
React ISP admin	Customers, plans, access accounts, subscriptions, payments, sessions, routers, reports and settings.
Nexacore platform admin	Organization onboarding, platform subscriptions, tenant support, system health and cross-tenant operations.
Hotspot captive portal	Package selection, phone capture, STK initiation/status, login and entitlement flow.
Flask API	Authentication, tenant scoping, billing rules, APIs, M-PESA, orchestration, audit and reporting.
PostgreSQL	Tenant business data, reporting aggregates and optionally RADIUS SQL schema.
FreeRADIUS	PPPoE/Hotspot authentication, authorization, accounting and session history.
Provisioning adapters	Translate business entitlement into RADIUS policy, Static IP, DHCP, queues or router configuration.

MikroTik RouterOS	Network enforcement, PPPoE, Hotspot, DHCP, queues, RADIUS client and accounting source.
Celery + Redis	Reconciliation, expiry, disconnects, notifications, usage aggregation and health checks.
Monitoring stack	API, server, router, RADIUS, queue and payment health visibility.

7. Technology Stack

Layer	Recommended technology	Reason
Backend	Python 3 + Flask	Preferred developer stack; explicit and controllable.
API	Flask Blueprints + schema validation	Modular domain boundaries.
ORM	SQLAlchemy 2.x	Relational mapping and transactional control.
Migrations	Alembic / Flask-Migrate	Repeatable schema evolution.
Database	PostgreSQL	Transactions, indexing, reporting and JSON.
Layer	Recommended technology	Reason
AAA	FreeRADIUS	Mature authentication/accounting.
Queue	Celery + Redis	Retries and scheduled jobs.
Frontend	React + Vite + Tailwind	Current web admin stack.
Captive portal	React/web portal	Mobile-first Hotspot purchase and login.
Reverse proxy	Nginx	TLS and request routing.
App server	Gunicorn	Production WSGI.
Deployment	Docker Compose initially	Reproducible environment.
Monitoring	Prometheus/Grafana + error tracking	Operational visibility.

8. Multi-Tenancy and Organization Model

Organization is the central SaaS boundary. Every ISP receives an Organization record. Tenant-owned users, customers, sites, routers, service plans, access accounts, subscriptions, payment intents, payments, accounting sessions, reports and audit events are tied to that organization.

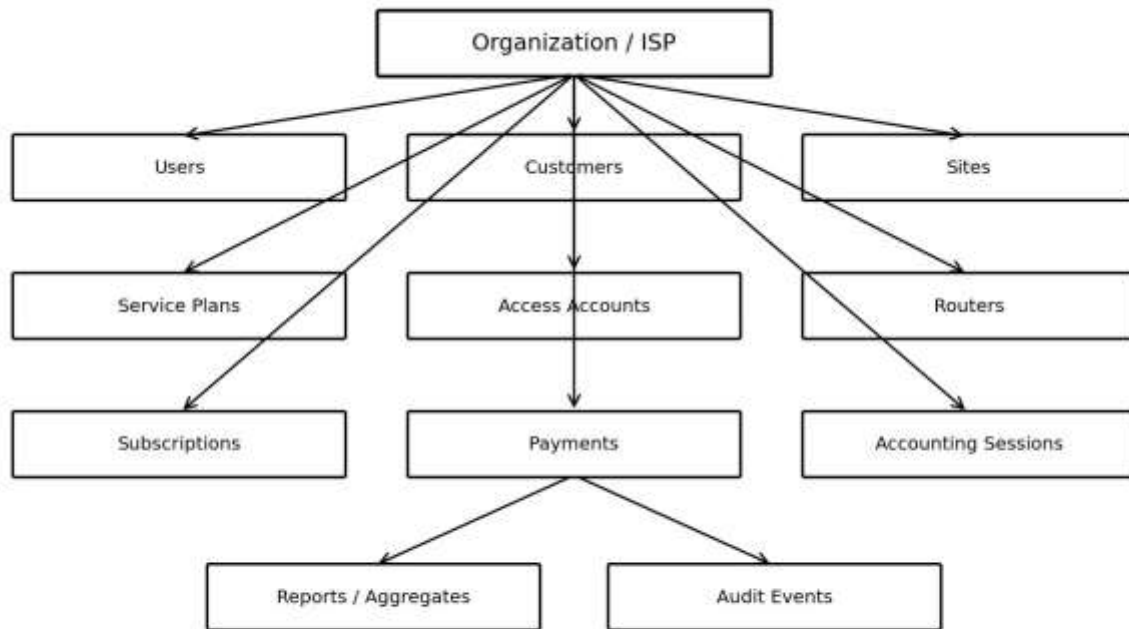


Figure 3. Tenant-owned domain hierarchy.

8.1 Organization fields

Field	Purpose
id	Primary tenant identifier.

name	Legal or trading name.
slug	Stable tenant slug; optional future subdomain key.
status	ACTIVE, SUSPENDED, CLOSED.
timezone	Display/report timezone; default Africa/Nairobi where appropriate.
contact_email / phone	Business contact.
branding_json	Future logo/colors/custom portal branding.
created_at / updated_at	Audit timestamps.

8.2 Tenant isolation rules

1. Authenticated tenant users receive an organization context from the server; the client cannot choose arbitrary organization_id values.
2. Every repository/query for tenant data filters by organization_id in addition to record ID.
3. Composite uniqueness is tenant-aware where appropriate, for example customer phone or router identity may be unique within one organization rather than globally.
4. Platform super admins may cross tenants only through explicit platform-admin routes and audited actions.
5. Background jobs carry organization_id and never infer tenant ownership from untrusted payload fields.
6. Reports, cached data, object storage paths and logs must preserve tenant boundaries.

8.3 Tenant-owned configuration

Each organization can eventually own its own M-PESA profile, SMS/email settings, router credentials, portal branding, invoice settings, and operational policies. Sensitive credentials must be encrypted or stored in a secrets manager; they must not be returned to ordinary API responses.

9. Identity, Roles and Authorization

9.1 Current development roles

During the two-developer build phase, both developers are platform super_admin users. This keeps implementation simple while preserving the role field for future evolution.

9.2 Target roles

Role	Scope	Typical permissions
PLATFORM_SUPER_ADMIN	Cross-tenant	Organizations, platform subscriptions, tenant support, system settings and health.
ORG_OWNER	One organization	All ISP operations and organization settings.
ADMIN (later)	One organization	Customers, services, payments, plans, sessions and reports.
TECHNICIAN (later)	One organization	Routers, sessions, installations and customer service actions; limited finance.
FINANCE/CASHIER (later)	One organization	Payments, customer lookup, receipts and finance reports.
READ_ONLY (later)	One organization	Dashboards and reports only.

Authorization must always be enforced server-side. Frontend role checks only control presentation and must never be treated as security.

10. Domain Model and Data Architecture

The business model remains independent from FreeRADIUS tables. RADIUS is an integration target, not the commercial data model. This allows Nexacore to support non-RADIUS services such as Static IP or DHCP and to replace or replicate AAA components without corrupting billing history.

Entity	Purpose	Important fields
Organization	ISP tenant	id, name, slug, status
User	Platform/ISP operator	organization_id nullable for platform users, email, role, active
Customer	Subscriber/person/business	organization_id, code, name, phone, email, status
Site	Physical service area/POP/building	organization_id, name, type, location, status
Router	Managed NAS/MikroTik	organization_id, site_id, identity, mgmt_ip, nas_ip, status

ServicePlan	Commercial/network package	organization_id, access_types, price, speed, validity, quotas
AccessAccount	Network identity independent from customer	organization_id, customer_id, type, username/identity, status
Subscription	Entitlement to plan over time	organization_id, account_id, plan_id, starts_at, expires_at, status
PaymentIntent	Requested payment	organization_id, customer_id, amount, purpose, provider IDs, state
Payment	Finalized receipt	organization_id, receipt_no, amount, msisdn, paid_at
AccountingSession	Network session projection	organization_id, account_id, router_id, session_id, traffic, IP/MAC
UsageAggregate	Daily/hourly rollup	organization_id, account/router/date, up/down bytes
AuditEvent	Immutable change trail	organization_id, actor, action, entity, before/after, timestamp

10.1 Status separation

Domain	Example states	Meaning
Customer	ACTIVE, INACTIVE, CLOSED	Commercial relationship.
AccessAccount	PENDING, ACTIVE, SUSPENDED, DISABLED	Whether the network identity is enabled.
Subscription	PENDING, ACTIVE, SUSPENDED, EXPIRED, CANCELLED	Whether service entitlement is valid.
Network session	ONLINE, OFFLINE, STALE	Actual connection state.
Router	ONLINE, DEGRADED, OFFLINE, DISABLED	Network-device health.

An active customer can be offline. An active access account can have an expired subscription. These states must never be collapsed into one overloaded status field.

11. Customer Management

11.1 Current API baseline

Method	Endpoint	Purpose
GET	/api/v1/customers	List with pagination, search and status filter.
POST	/api/v1/customers	Create customer.
GET	/api/v1/customers/{id}	Customer detail.
PATCH	/api/v1/customers/{id}	Update customer.
PATCH	/api/v1/customers/{id}/status	Activate/suspend/inactivate customer as policy allows.

11.2 Tenant upgrade

After Organization is implemented, every Customer receives organization_id and all customer endpoints are automatically scoped to the authenticated ISP. A customer ID from another ISP must behave as not found or forbidden according to security policy and must never leak record existence unnecessarily.

11.3 Customer detail view

- Identity and contact information.
- Current access accounts by type.
- Current and historical subscriptions.
- Payment history and M-PESA receipts.
- Online/recent sessions and usage.
- Assigned site/router where applicable.
- Tickets and communication history when those modules are added.  Audited administrative actions.

12. Access Services: PPPoE, Hotspot, Static IP and DHCP

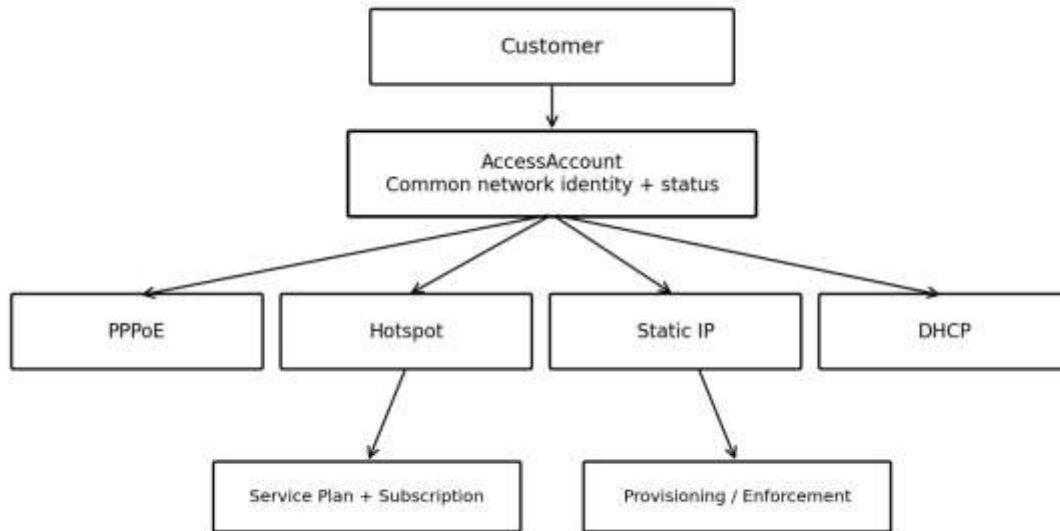


Figure 4. A generic AccessAccount supports multiple network access methods without redesigning customer billing.

12.1 Generic AccessAccount

AccessAccount is the bridge between commercial entitlement and network identity. Common fields include organization_id, customer_id, router/site scope, connection_type, username or device identity, status, and lifecycle timestamps. Protocolspecific details belong in dedicated tables or structured policy data.

Access type	Primary identity	Typical enforcement / integration
PPPoE	Username + password	FreeRADIUS authentication; MikroTik PPPoE server; rate attributes; accounting.
Hotspot	Username/token/phone and optional device	MikroTik HotSpot + FreeRADIUS; captive portal; time/data/session policy.
Static IP	Assigned IP/CPE/device	Router provisioning, address lists, queues, ARP/firewall policy as designed; RADIUS optional.
DHCP	MAC/client ID/reservation	DHCP reservation/lease plus policy/queue/address-list enforcement; RADIUS only where supported/appropriate.

12.2 PPPoE flow

1. Create customer and PPPoE AccessAccount.
2. Assign ServicePlan and Subscription.
3. Synchronize RADIUS policy.
4. Configure subscriber CPE with credentials.
5. MikroTik sends Access-Request to FreeRADIUS.
6. FreeRADIUS validates active entitlement and returns rate/session policy.
7. Start/Interim/Stop accounting feeds session and usage data.

12.3 Hotspot flow

1. User joins Hotspot SSID and receives DHCP address.
2. Captive portal presents plans.
3. User provides phone and initiates STK Push.
4. Backend verifies payment finality.
5. AccessAccount/token and Subscription are created or renewed.
6. FreeRADIUS authorizes session and applies limits.

12.4 Static IP and DHCP design rule

Static and DHCP access must reuse the same Customer -> AccessAccount -> Subscription -> ServicePlan commercial flow. The difference is only the network provisioning adapter. This prevents duplicated customer/payment logic and makes reporting comparable across access technologies.

13. Sites, Routers and Network Inventory

13.1 Site

A Site represents a physical service area, building, market, POP, estate, school, business premises, or other deployment location. Sites belong to one Organization and may contain one or more routers.

13.2 Router/NAS

Field	Purpose
organization_id	Tenant ownership.
site_id	Physical deployment relationship.
name / identity	Human and RouterOS identifiers.
mgmt_ip	Private management endpoint.
nas_ip / nas_identifier	RADIUS identity.
routeros_version	Compatibility/health.
capabilities	PPPoE, Hotspot, DHCP, API, CoA, etc.
status / last_seen_at	Operational state.
secret references	Encrypted/secret-manager pointers only.

13.3 Router filtering

Dashboards and reports should support All Routers and a specific-router filter. Revenue, online users, traffic, top consumers, alerts and router health can then be viewed organization-wide or per network edge.

13.4 Future network inventory

- OLT vendor/device management.
- Chained router / bridge monitoring.
- TR-069 ACS / CPE management.
- Switch/AP inventory.
- Fiber/port/ONU relationships as a later domain.

14. Service Plans, Subscriptions, FUP and Burst

14.1 ServicePlan

Field	Example	Meaning
access_types	PPPOE/HOTSPOT/STATIC/DHCP	Where the plan is allowed.
price_kes	1500	ISP selling price.
validity_mode	ABSOLUTE / ONLINE_TIME / DATA	How entitlement is consumed.
validity_seconds	2592000	Example 30 days.
upload_bps	5000000	Upload ceiling.
download_bps	10000000	Download ceiling.
data_cap_bytes	NULL	Optional data cap.
max_sessions	1	Concurrent session limit.
fup_policy_id	optional	Fair-use threshold and throttle policy.
burst_policy_id	optional	Burst rate/time/threshold policy.

14.2 Subscription renewal

- If active, extend according to the configured renewal policy.
- If expired, normally begin from verified payment time.
- Manual extensions require actor, reason, previous expiry, new expiry and audit event.
- A successful payment must activate/renew exactly once.

14.3 FUP

Fair Use Policy is a later capability built on accounting aggregates. A plan may define a usage threshold per period and a reduced rate after the threshold. The system must preserve the original plan and record FUP state transitions rather than silently modifying subscriber speed.

14.4 Burst

Burst policies may define burst upload/download rates, burst threshold and burst time for compatible MikroTik enforcement. Burst must be treated as plan policy, not manually embedded into customer records.

15. FreeRADIUS and AAA

FreeRADIUS provides SQL-backed authentication, authorization and accounting for PPPoE and Hotspot. Nexacore owns commercial entities and derives the RADIUS rows/policy required for network access.

RADIUS table	Purpose
radcheck	Per-user checks and credentials.
radreply	Per-user reply attributes.
radusergroup	Account-to-policy group mapping.
radgroupreply	Shared rate/session attributes.
radgroupcheck	Shared authorization constraints.
radacct	Start/Interim/Stop accounting records.
radpostauth	Authentication result history where enabled.

15.1 Accounting interval

Configure Start, Interim-Update and Stop accounting. A five-minute starting interval is reasonable for MVP visibility, but the final interval must be validated against database write volume and required quota accuracy.

15.2 CoA and disconnect

Use RADIUS Disconnect or Change of Authorization where MikroTik and policy support it. Expiry/suspension may require immediate disconnect. Speed changes may use CoA if the attributes can be changed in-session; otherwise disconnect and reconnect.

16. MikroTik and Provisioning Integration

16.1 Router registration

- Organization and site ownership.
- Router identity and management IP.
- RADIUS NAS IP / NAS-Identifier.
- RouterOS version and capabilities.
- Encrypted API/SSH credentials or external secret reference.
- Last-seen, CPU, memory, uptime and storage health.

16.2 Provisioning adapters

Adapter	Responsibilities
RadiusProvisioner	PPPoE/Hotspot credentials, groups, rate attributes and state.
StaticProvisioner	IP assignment, address lists, queues/firewall and customer mapping as network design requires.
DhcpProvisioner	Reservations/leases, MAC/client identity and related policy.
RouterActionService	Disconnect, CoA, queue/profile changes and safe retries.

16.3 Safety rules

- Payments never fail because a router is temporarily unreachable; provisioning becomes a retryable operational error. ☒
Destructive router actions require authenticated operator context and audit event.
- Router management services should be private/VPN restricted.
- Use strong unique RADIUS secrets per NAS where feasible.

17. Sessions, Accounting and Traffic Analytics

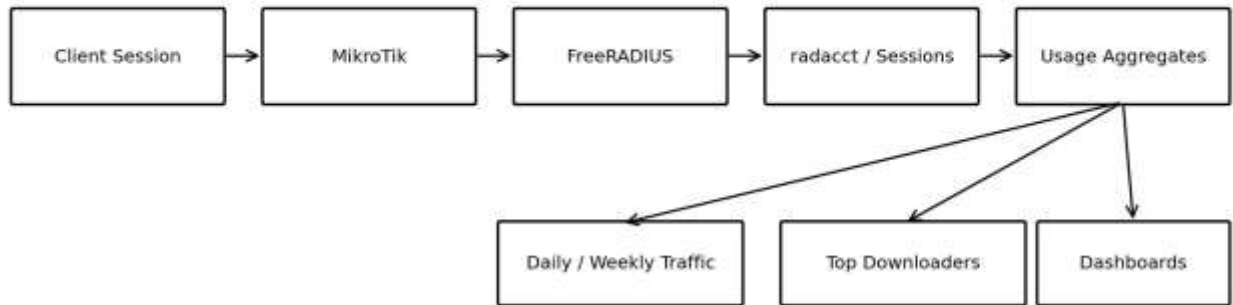


Figure 5. Accounting data is rolled into daily/weekly analytics rather than scanning raw sessions on every dashboard request.

17.1 AccountingSession

Field	Purpose
organization_id	Tenant scope.
access_account_id	Subscriber network identity.
router_id	NAS / edge source.
session_id	RADIUS/session correlation.
started_at / ended_at	Session duration.
ip_address / mac_address	Observed addressing/device data.
upload_bytes / download_bytes	Usage counters.
session_seconds	Connected duration.
state	ONLINE/OFFLINE/STALE projection.

17.2 Required traffic views

- Daily network traffic.
- Weekly network traffic.
- Monthly usage trend.
- Top downloaders / top uploaders.
- Usage per customer.
- Usage per access account.
- Usage per router and per site.
- Organization-wide traffic totals.

17.3 Aggregation rule

Dashboards must not repeatedly scan the full raw radacct history. Scheduled aggregation rolls accounting records into hourly/daily summaries. Raw accounting remains available for diagnostics and detailed history, while dashboards read indexed aggregates.

18. M-PESA and Payments

Payment design retains the verified-finality model from v1.0. Each organization may eventually own its own configured M-PESA profile. Payment credentials are tenant-sensitive secrets and require encryption or secret-manager storage.

18.1 STK lifecycle

```

CREATED -> STK_REQUESTED -> PENDING_CUSTOMER -> PAID
                                     -> FAILED
                                     -> CANCELLED
-> TIMED_OUT
                                     -> UNKNOWN -> RECONCILED
  
```

18.2 Callback rules

- Locate PaymentIntent by provider correlation IDs.
- Process inside a database transaction.
- Enforce unique receipt/reference constraints.
- If already PAID, acknowledge without renewing twice.
- Do not trust client-side success messages.
- Support delayed callback and reconciliation query.
- Never expose Daraja credentials or sensitive raw logs to clients.

18.3 Payment configuration ownership

A future OrganizationPaymentProfile may reference BusinessShortCode, Till/PartyB, callback configuration and secret identifiers. Public tenant APIs may expose masked metadata only. Platform staff access to payment configuration must be tightly audited.

19. Finance, Reports and Revenue Analytics

19.1 Core finance views

- Income today.
- Income this month.
- Payments by period.
- Recent M-PESA transactions.
- Outstanding/overdue invoices where invoicing is enabled.
- Revenue by router/site/plan/access type.
- Subscriber growth and churn.
- Manual payment adjustments with audit trail.

19.2 Later finance modules

- Expenditure tracking.
- Sales commissions.
- Inventory valuation.
- Tax/invoice extensions.
- Advanced profit and loss analytics.

Financial records should be append-only wherever practical. Corrections use reversal or adjustment records rather than silent edits.

20. ISP Dashboard and User Experience

The ISP-facing dashboard should translate operational state into clearly named, non-overlapping metrics. Counters must state exactly what they represent to avoid ambiguity between customers, access accounts, subscriptions and sessions.

Dashboard area	Recommended metrics
Revenue	Income today, income this month, payment count, recent transactions.
Dashboard area	Recommended metrics
Subscribers	Registered customers, active subscriptions, expired/suspended subscriptions.
Connection	PPPoE online, Hotspot online, total online sessions, offline/stale sessions.
Access accounts	PPPoE total, Hotspot total, Static total, DHCP total.
Support	Open, pending and solved tickets - later release.
Network	Router status, CPU, memory, uptime, storage, site alerts.
Traffic	Daily traffic, weekly traffic, top consumers, per-router filter.
Communication	SMS/email queued/sent counts - later release.
Billing exceptions	Overdue invoices, failed STK, pending reconciliation, provisioning errors.

20.1 Router selector

At the top of network/revenue dashboards, the operator should be able to select All Routers or one specific router. The selected context filters online users, traffic, health, usage and relevant revenue/service metrics.

20.2 Main navigation

Group	Modules
Main	Dashboard
Clients & Support	Customers, Prepaid/Hotspot accounts, Tickets, Escalations, Communication
Plans & Usage	Service Plans, Subscriptions, FUP, Burst, Calendar
Payments & Finance	Payments, Finance, Expenditure, Commissions, Reports, Analytics
Network	Sites, Routers, RADIUS, Sessions, OLT, CPE/TR-069, chainedrouter monitoring
Operations	Inventory, payment links, portal/static pages
System	Data integrity, downtime compensation, STK audit, logs, documentation, settings

Not all navigation items belong in the first release. The grouping is a target information architecture that allows later modules to be added without redesigning the entire dashboard.

21. Platform Administration

The Nexacore platform dashboard is different from the ISP dashboard. Platform super admins manage SaaS tenants, not subscriber network operations by default.

- Create/suspend organizations.
- Assign organization owners.
- View platform subscription status.
- Monitor platform services, database, workers, RADIUS infrastructure and global incidents.
- Provide audited tenant support tooling.
- View aggregate platform usage without exposing one tenant to another. Manage feature flags and deployment configuration.

22. Support, Communication and Operations

22.1 Tickets and escalation

Later releases may provide customer support tickets with OPEN, PENDING, SOLVED and ESCALATED states, assignment, comments and SLA timing. Network/service context can be linked to the ticket.

22.2 Communication

SMS and email modules can send expiry notices, payment confirmations, outage updates and support messages.

Communication logs must be tenant-scoped and should capture delivery state, template, recipient and event source. **22.3 Data**

integrity and audits

- STK prompt audit to trace initiated payment attempts.
- Data-integrity checks for orphaned subscriptions/accounts and duplicate identities.
- System logs with request and tenant context.
- Downtime compensation as a later policy-driven subscription adjustment.

23. Flask Backend Architecture

23.1 Recommended modular layout

```
backend/
  app/
    __init__.py
  config.py
  extensions.py
```

```

authz.py      api/
auth.py
organizations.py
customers.py   sites.py
routers.py
service_plans.py
access_accounts.py
subscriptions.py
payments.py
sessions.py
reports.py     models/
organization.py
user.py        customer.py
site.py        router.py
service_plan.py
access_account.py
subscription.py
payment.py
accounting_session.py
    services/
tenant_service.py
subscription_service.py
payment_service.py
provisioning_service.py
usage_service.py
integrations/    daraja.py
radius.py        mikrotik.py
common/ migrations/
    tests/
wsgi.py
worker.py

```

23.2 Layering

Layer	Rule
Routes/controllers	HTTP only: validation, auth, tenant context, status codes and serialization.
Services	Business transactions: subscription lifecycle, payments, provisioning, reporting.
ORM/repositories	Database access with explicit tenant scoping.
Integrations	Daraja, MikroTik, RADIUS and future OLT/CPE adapters behind interfaces.
Workers	Call services and preserve tenant context; do not duplicate policy logic.

24. API Design

All APIs are versioned under `/api/v1`. Tenant-owned endpoints derive tenant context from the authenticated user. Pagination is required for list endpoints. Timestamps use ISO 8601 UTC in storage/API and are rendered in the organization timezone in the UI.

Method	Endpoint	Purpose
POST	<code>/auth/login</code>	Operator authentication.
GET	<code>/auth/me</code>	Current user and organization context.
GET/POST	<code>/organizations</code>	Platform-admin tenant management.
POST	<code>/organizations/{id}/owners</code>	Create ISP owner.
GET/POST	<code>/customers</code>	Tenant customer list/create.
GET/PATCH	<code>/customers/{id}</code>	Tenant customer detail/update.
GET/POST	<code>/sites</code>	Site management.
GET/POST	<code>/routers</code>	Router/NAS inventory.
GET/POST	<code>/service-plans</code>	Internet service-plan management.
POST	<code>/access-accounts</code>	Create PPPoE/Hotspot/Static/DHCP account.
POST	<code>/subscriptions</code>	Create entitlement.
POST	<code>/subscriptions/{id}/suspend</code>	Suspend service.
POST	<code>/payments/stk</code>	Create payment intent + STK.
POST	<code>/mpesa/stk/callback</code>	Public Daraja callback; no user JWT.
GET	<code>/sessions?online=true</code>	Online sessions.
GET	<code>/reports/traffic</code>	Daily/weekly usage analytics.
GET	<code>/reports/revenue</code>	Revenue summary.

24.1 API error shape

```
{
  "status": "error",
  "error": {
    "code": "TENANT_ACCESS_DENIED",
    "message": "The requested resource is not available.",
    "request_id": "req_..."
  }
}
```

25. Background Jobs and Automation

Job	Trigger	Action
expire_subscriptions	Scheduled	Mark expired, update policy and disconnect if required.
reconcile_mpesa	Scheduled	Query stale pending STK transactions.
send_expiry_notices	Scheduled	Notify customers before/after expiry.
router_actions	Event driven	Disconnect/CoA/provision with retry.
aggregate_usage	Scheduled	Roll session accounting into hourly/daily summaries.
router_health	Scheduled	Poll reachability, CPU, memory, uptime and RADIUS health.
cleanup_stale_sessions	Scheduled	Close stale sessions conservatively.
tenant_usage_rollup	Scheduled	Compute organization-level SaaS usage and limits.
Job	Trigger	Action
backup_verify	Daily	Verify backup completion/restoration metadata.

26. Security and Tenant Isolation

26.1 Application security

- HTTPS everywhere; HSTS after validation.
- Short-lived JWT access tokens with refresh-token strategy.
- Strong password hashing.
- Server-side role and tenant authorization.
- Rate limiting on login, payment initiation and public portal endpoints.
- Strict JSON validation and parameterized ORM queries.
- Controlled CORS origins.
- Audit sensitive actions and selected sensitive reads.

26.2 Tenant security

- Never accept organization_id from a tenant client as authorization proof.
- All tenant table access requires organization scope.
- Use tenant-aware unique constraints and indexes.
- Partition/cache keys include organization ID.
- Object storage and exports include tenant-scoped paths/permissions.
- Cross-tenant platform-support actions are explicit and audited.

26.3 Secrets

- Never commit Daraja keys, passkeys, JWT keys, RADIUS secrets or router credentials.
- Use separate local/staging/production secrets.
- Encrypt tenant payment/router credentials at rest or use a secrets manager.
- Rotate credentials with documented procedures.

26.4 Kenya data protection

Nexacore processes names, phone numbers, IP/MAC/device data, session history and payment references. Apply data minimization, retention, access controls, breach procedures and applicable Kenya Data Protection Act / ODPC requirements. Obtain legal/compliance review before commercial production.

27. Reliability and Failure Handling

Failure	Expected behavior
Daraja request timeout	Keep intent unresolved; reconcile before a new charge where possible.
Duplicate successful callback	Unique provider IDs + transaction locks make duplicate processing a no-op.
Callback missing	Scheduled query; unresolved items enter manual review.
FreeRADIUS unavailable	Existing sessions may continue; new AAA may fail; alert immediately.
Database unavailable	Fail safely; do not finalize payments that cannot be recorded.
Router unreachable after payment	Payment remains valid; provisioning retries and operator sees error.
Accounting Stop lost	Use interim data and stale-session reconciliation.
Worker outage	Core API reads continue where safe; queued jobs resume after recovery.
Tenant configuration error	Contain impact to that organization; do not affect other tenants.

27.1 Transactional outbox

For high-value events, write outbox records in the same database transaction as subscription/payment changes. Workers publish router actions and notifications from the outbox so a committed business change cannot silently lose its external side effect.

28. Observability, Auditing and Health

28.1 Required metrics

- API latency/error rate by tenant-safe labels.
- STK requested/success/failure/pending counts.
- RADIUS accepts/rejects/timeouts.
- Accounting update lag and active sessions.
- Router/site online/offline/degraded state.
- Router CPU, memory, storage and uptime.
- Server CPU, RAM, disk, database and queue health.
- Daily/weekly traffic and top-consumer aggregates.
- Background-job queue depth and retry failures.

28.2 Audit context

Structured logs and AuditEvent should carry request_id, actor_id, organization_id, site_id/router_id where relevant, entity type/id, action and timestamp. Payment logs may include provider IDs but never secrets or PIN information.

29. Backup and Disaster Recovery

- Automated PostgreSQL backups daily; WAL/PITR if budget allows.
- Encrypted off-server backup copy.
- Back up FreeRADIUS, Nginx, Docker, firewall and critical RouterOS configuration exports.
- Quarterly restore test into an isolated environment.
- Define RPO/RTO before commercial launch.
- Test tenant-level export/recovery procedures if per-tenant restore becomes a requirement.

30. Deployment and Environment Strategy

30.1 Environments

Environment	Purpose	External integrations
Local	Developer WSL/Docker	Mocks; optional Daraja sandbox; lab router.
Staging	Production-like multi-tenant validation	Daraja sandbox/test; test routers/RADIUS.
Production	Live ISP organizations	Production M-PESA, live routers and monitored services.

30.2 Initial production services

```
nginx flask-
api celery-
worker
celery-beat
postgres
redis
freeradius
wireguard / private management overlay
monitoring stack
```

30.3 Tenant deployment choice

The target is shared SaaS with row-level application tenancy. Larger enterprise customers may later require dedicated deployment or database isolation, but the core domain should still retain Organization ownership so both deployment models remain possible.

31. Testing Strategy

31.1 Unit tests

- Phone normalization.
- Tenant scope filters and cross-tenant access denial.
- Subscription renewal calculations and state transitions.
- Plan-to-network policy mapping.
- PPPoE/Hotspot RADIUS policy generation.
- Static/DHCP provisioning payload generation.
- Payment callback parser and idempotency.
- Usage aggregation.  Authorization roles.

31.2 Integration tests

- Flask + PostgreSQL transaction tests.
- Organization A cannot read/update Organization B records.
- FreeRADIUS SQL authentication and accounting.
- MikroTik lab router PPPoE login.
- Hotspot portal + RADIUS login.
- Static/DHCP provisioning on a lab router.
- Daraja sandbox STK and callback simulation.
- CoA/disconnect and retry behavior.

31.3 End-to-end tests

Scenario	Expected result
New ISP onboarding	Organization + owner created; owner only sees own tenant.
New PPPoE subscriber	Create -> subscribe -> authenticate -> rate -> accounting visible.
Expired subscriber	Entitlement expires -> access denied/suspended -> session handled.
M-PESA renewal	Payment finalized once -> subscription extended once -> service restored.
Duplicate callback	No duplicate payment or renewal.
Hotspot purchase	STK -> entitlement -> login -> expiry enforced.
Static/DHCP subscriber	Subscription change produces correct router policy.
Cross-tenant ID attack	No record disclosure or modification.
Router offline at renewal	Payment recorded; provisioning retry visible.
Traffic report	Daily/weekly totals reconcile to accounting aggregates within defined tolerance.

32. Performance and Scaling

- Index every tenant-owned table beginning with organization_id and common filter columns.
- Use composite indexes for organization_id + status/date/foreign key patterns.
- Do not build dashboards by scanning full raw accounting tables.
- Use connection pools sized to PostgreSQL capacity.
- Cache stable plan/router metadata but never use stale cache as payment finality.
- Scale Flask and Celery horizontally behind Nginx as load grows.

- Add redundant FreeRADIUS nodes for larger deployments.
- Measure accounting update rate before reducing interim intervals.
- Consider table partitioning for large accounting/payment event histories.

33. Implementation Roadmap

Phase	Deliverable	Status
0	Flask + PostgreSQL + migrations + health checks	Complete
1	JWT authentication + platform super-admin flow	Complete baseline
2	Customer CRUD and search	Complete baseline
Phase	Deliverable	Status
3	Organization model + tenant isolation + ISP owner	NEXT
4	Sites + Routers/NAS	Planned
5	Service Plans + generic AccessAccount	Planned
6	PPPoE + FreeRADIUS AAA/accounting	Planned
7	Subscriptions + expiry/suspend/restore	Planned
8	M-PESA STK + reconciliation + renewals	Planned
9	Hotspot captive portal + prepaid flows	Planned
10	Static IP + DHCP provisioning adapters	Planned
11	Traffic aggregation + dashboards + top consumers	Planned
12	Production hardening, audit, backups, monitoring	Planned
13	Tickets, communication, FUP, burst, finance extras	Later
14	OLT, TR-069 CPE, inventory, downtime compensation	Later

33.1 Commercial MVP exit gate

Do not treat Nexacore as commercially ready until at least two test organizations can be onboarded, tenant isolation is proven, a complete subscriber lifecycle works, verified M-PESA payment renews exactly once, network access is enforced correctly, usage/accounting is visible, and one organization cannot discover or manipulate another organization's data.

34. Acceptance Criteria

- Every tenant-owned API is organization-scoped server-side.
- Cross-tenant record access fails safely.
- A customer can own one or more AccessAccounts of supported types.
- PPPoE/Hotspot active subscriptions authenticate with correct policy.
- Static/DHCP subscriptions produce correct network provisioning for supported deployments.
- Expired/suspended entitlement prevents normal access according to policy.
- RADIUS Start/Interim/Stop accounting is visible and aggregated.
- Daily and weekly traffic reports are available without expensive raw-table scans.
- Duplicate payment callbacks do not duplicate renewal.
- Router outages do not corrupt payment records.
- Operators can identify failed login/payment/provisioning reasons without exposing secrets.
- Backups can be restored.

35. Risks and Open Decisions

Decision / risk	Current position
Tenant model	Adopt Organization now before adding Sites/Routers/Plans.
Current admin_user table	May evolve/rename to general users; platform users have no tenant, ISP users belong to organization.
Per-ISP M-PESA credentials	Design for tenant-specific payment profiles; exact commercial onboarding model still to be decided.

PPPoE passwords	Use random secrets; protect any reversible RADIUS credential storage tightly.
Static/DHCP enforcement	Define supported router templates and avoid pretending all networks use the same mechanism.
Hotspot timers	Use absolute validity first; online-time/data counters later.
FUP/Burst	Plan-driven later release built on usage/accounting.
Subdomains/branding	Optional future capability; not required for first tenant isolation.
OLT/CPE scope	Later release after billing/network-access core is stable.
Platform billing	Keep SaaS PlatformSubscription separate from ISP ServicePlan.
Single vs dedicated deployments	Shared SaaS is target; architecture should permit dedicated enterprise deployments later.

Appendix A. Target Database Schema

```

organizations(id, name, slug, status, timezone, contact_email, contact_phone, created_at,
updated_at)
users(id, organization_id NULLABLE, full_name, email, password_hash, role, is_active, created_at,
last_login_at)
customers(id, organization_id, customer_code, full_name, phone, email, address, notes, status,
created_at, updated_at)
sites(id, organization_id, name, site_type, location, timezone, status, created_at, updated_at)
routers(id, organization_id, site_id, name, identity, mgmt_ip, nas_ip, status, capabilities_json,
last_seen_at)
service_plans(id, organization_id, name, price_kes, validity_mode, validity_seconds, upload_bps,
download_bps, data_cap_bytes, max_sessions, fup_policy_id, burst_policy_id, active)
access_accounts(id, organization_id, customer_id, site_id, router_id, type, username_or_identity,
secret_encrypted, status, created_at)
static_assignments(id, organization_id, access_account_id, ip_address, gateway, prefix,
policy_json)
dhcp_assignments(id, organization_id, access_account_id, mac_address, client_id, ip_address,
policy_json)
subscriptions(id, organization_id, access_account_id, service_plan_id, starts_at, expires_at,
status, source_payment_id, created_at)
payment_intents(id, organization_id, customer_id, service_plan_id, amount_kes, phone, provider,
state, merchant_request_id, checkout_request_id, idempotency_key, created_at, finalized_at)
payments(id, organization_id, intent_id, receipt_no, amount_kes, phone, paid_at,
raw_reference_json, created_at)
accounting_sessions(id, organization_id, access_account_id, router_id, session_id, started_at,
ended_at, upload_bytes, download_bytes, ip_address, mac_address, state)
usage_aggregates(id, organization_id, access_account_id NULLABLE, router_id NULLABLE, bucket_date,
upload_bytes, download_bytes, session_seconds)
audit_events(id, organization_id NULLABLE, actor_id, action, entity_type, entity_id, before_json,
after_json, request_id, created_at)
outbox_events(id, organization_id NULLABLE, event_type, payload_json, state, attempts,
next_attempt_at, created_at) platform_subscriptions(id, organization_id, plan_code, status,
starts_at, renews_at, limits_json)

```

FreeRADIUS schema: radcheck, radreply, radusergroup, radgroupcheck, radgroupreply, radacct, radpostauth

Appendix B. Suggested Status Enums

Domain	States
Organization	ACTIVE, SUSPENDED, CLOSED
User	ACTIVE, DISABLED
Customer	ACTIVE, INACTIVE, CLOSED
AccessAccount	PENDING, ACTIVE, SUSPENDED, DISABLED
Subscription	PENDING, ACTIVE, SUSPENDED, EXPIRED, CANCELLED
PaymentIntent	CREATED, STK_REQUESTED, PENDING_CUSTOMER, PAID, FAILED, CANCELLED, TIMED_OUT, UNKNOWN
Router	ONLINE, DEGRADED, OFFLINE, DISABLED
Session	ONLINE, OFFLINE, STALE
Ticket - later	OPEN, PENDING, SOLVED, ESCALATED
Outbox	PENDING, PROCESSING, DONE, FAILED

Appendix C. RADIUS Attributes

Attribute	Use
User-Name	Access-account username.
Calling-Station-Id	Client MAC/device identity as reported by NAS.
NAS-IP-Address / NAS-Identifier	Router/NAS identity.
Acct-Session-Id	Session correlation.
Acct-Status-Type	Start, Stop, Interim-Update.
Acct-Session-Time	Online seconds.
Acct-Input/Output-Octets + Gigawords	Traffic counters; direction interpreted carefully.
Mikrotik-Rate-Limit	Subscriber bandwidth policy.
Attribute	Use
Session-Timeout	Session lifetime where appropriate.
Idle-Timeout	Idle logout.
Mikrotik-Group	Optional profile/policy mapping.

Appendix D. MikroTik Baseline

```
# RADIUS client example
/radius add service=ppp,hotspot address=<RADIUS_PRIVATE_IP> secret=<UNIQUE_SECRET>
authenticationport=1812 accounting-port=1813

# PPP remote AAA
/ppp aaa set use-radius=yes accounting=yes interim-update=5m

# Hotspot profile (replace <profile>)
/ip hotspot profile set <profile> use-radius=yes radius-accounting=yes radius-
interimupdate=received

# Security baseline
# Restrict API/SSH/Winbox to management VPN/subnet and disable unused services.
# Static/DHCP provisioning templates are deployment-specific and must be validated in a lab.
```

Appendix E. M-PESA Request and Callback Rules

```
Payment intent -> STK request -> customer prompt -> callback/query -> verified payment ->
subscription transaction -> provisioning/outbox
```

Critical guarantees:

- STK request acceptance is not payment success.
- Provider receipt/reference is unique.
- Callback processing is idempotent.
- Renewal happens exactly once.
- Payment remains valid if router provisioning temporarily fails.
- Tenant payment profile is resolved server-side from organization context.

Appendix F. Environment Variables

```
APP_ENV=production
DATABASE_URL=postgresql+psycopg://...
REDIS_URL=redis://...
JWT_SECRET_KEY=<secret>
FRONTEND_ORIGIN=https://...
DARAJA_BASE_URL=https://api.safaricom.co.ke
RADIUS_DB_URL=postgresql://...
ROUTER_CREDENTIAL_ENCRYPTION_KEY=<secret>
SENTRY_DSN=<optional>
```

Note: Tenant-specific M-PESA and router secrets should not be represented as global plaintext environment variables once multi-tenancy is enabled. Store encrypted tenant configuration or secret references.

Appendix G. Dashboard Metric Definitions

Metric	Definition
Registered Customers	Count of customer records in organization, not access accounts.
Active Subscriptions	Subscriptions currently entitled by date/state.
PPPoE Accounts	AccessAccount type PPPOE.
Hotspot Accounts	AccessAccount type HOTSPOT.
Online PPPoE	Current online sessions belonging to PPPoE accounts.
Metric	Definition
Online Hotspot	Current online sessions belonging to Hotspot accounts.
Revenue Today	Finalized payments with paid_at in organization-local current day.
Revenue This Month	Finalized payments within organization-local month.
Daily Traffic	Aggregated upload/download bytes for selected day/context.
Weekly Traffic	Aggregated upload/download bytes for seven-day reporting period.
Top Downloaders	Accounts ranked by aggregated download bytes for selected period/context.
Router Health	Reachability plus last-seen and device resource metrics.
Overdue	Invoice/subscription definition must be explicit; never use one label for multiple states.

Appendix H. Reference Sources

The original v1.0 engineering references remain applicable for RADIUS, MikroTik, M-PESA and Kenyan data-protection design. Exact production credentials, merchant mappings and device capabilities must always be verified against the specific provider/account/device configuration.

Reference	Used for
MikroTik RouterOS documentation - help.mikrotik.com/docs	RADIUS, PPPoE, HotSpot, DHCP, router management and accounting behavior.
FreeRADIUS documentation - freeradius.org/documentation	SQL authorization/accounting and standard RADIUS schema.
Safaricom Daraja developer portal - developer.safaricom.co.ke	M-PESA integration and production onboarding.
Kenya ODPC - odpc.go.ke	Data-protection responsibilities and compliance guidance.
Kenya Data Protection Act	Primary legal framework for personal-data processing in Kenya.

END OF SPECIFICATION